# CS & IT ENGINEERING


## Algorithms

### Analysis of Algorithms

Lecture No.- 12


By- Aditya sir

# Recap of Previous Lecture

**Topic** Time complexity of recursive Algo.

**Topic** Loop complexity

# Topics to be Covered

Topic Loop complexities Advanced

Topic Space complexity

# ABOUT ME

## Hello, I'm Aditya.

1. Represented college as the first Google DSC Ambassador.
2. The only student from the batch to secure an internship at Amazon.
3. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis8. Completed my Masters with an overall GPA of 9.36/108. Joined Dream11 as a Data Scientist.9. Have mentored around 10k working professions in field of Data Science and Analytics10. Have been mentoring GATE aspirants to secure a great rank in limited time.11. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.


**By- Aditya sir**

# Topic : Analysis of Algorithms

## Noted loops complexity

Loop within loop

2 types:-

1. Nested dependent loops
2. Nested independent loops

# Topic : Analysis of Algorithms

**Eg.1.**

```
a  = 0
for (i = 1; i <=n; i++)
{
        for (j =1; j <=n; j++)
        {
            a = a + 1
        }
}
```

$\longrightarrow O(n)$

nested independent loop

$\longrightarrow O(n)$

$\Rightarrow$ TC: $O(n^2)$

# Topic : Analysis of Algorithms

**Mutually Exclusive loops:**

(Not nested)

```
Algo AJ(n, m)
{
    for (i =1; i <=n ; i++)
    {
        print (i)
    }
    for (j = 1; j <=m; j++)
    {
        print (j)
    }
}
```

$\longrightarrow O(n)$

$\longrightarrow O(m)$

$\Rightarrow O(m+n)$

$= O(\max(m,n))$

# Topic : Analysis of Algorithms

41%

```
Algo AJ(n)
{
    for (i=1; i<=n; i++)
    {
        print (1)
    }
    for (i =1; i<=n; i++)
    {
        print (i+1)
    }
    for ((i=1; i<=n;i++)
        for (j =1, j <=n/5; j++)
            for (k =1; k <=n; k++)
            {
                break
            }
}
```

$\longrightarrow O(n)$

$\longrightarrow O(n)$

$O(n)$

$O(n)$

$O(1)$

$\rightarrow O(n^2)$

- A $O(n)$
- B $O(n^2)$
- C $O(n^3)$
- D $O(n^4)$

# Topic : Analysis of Algorithms

1.
```
For (i =1; i<=n; i =i+1)
{
    print(i)
}
```
→ n , O(n)

2.
```
For (i=1; i<=n; i=i+2)
{
    print(i)
}
```
→ n/2 , O(n)

3.
```
For (i=1; i<=n; i=i+7)
{
    print(i)
}
```
i = i+7

→ n/7 , O(n)

# Topic : Analysis of Algorithms

Generalized from

```
For (i=1; i<=n; i =i+b)
{
        print(i)
}
```

→ $n/b$ times

TC ⇒ $O(n)$

Sat & Sun
(28 & 29 June)

$\Rightarrow$ 12:30 - 2:30 PM

# Topic : Analysis of Algorithms

1. For (i=1; i<=n; i=i*2)

```
{
    print(i)
}
```

**Way-1**

$i=1, 1*2=2^1, 2^2, 2^3 \ldots\ldots 2^k$

Let assume loop runs for k form

$2^k = n$ (for last iterant )

$\boxed{k = \log_2 n}$

**Way-2**

Let n = 16

$i \Rightarrow \underbrace{2,4,8,16}_{4 times}$

$= \log_2 {}^{16}$

$= 4$

# Topic : Analysis of Algorithms

2. For (i=1; i<=n; i=i×5)

```
{
    print (i)
}
```

$5^k = n$

$k = \log_5 n$

$i = 1, 5^1, 5^2 \ldots\ldots 5^k$

$k^{th}$ itr ~~math~~ ; ~~$5^k = n$~~

(last) ~~$k = \log_5 n$~~

(iter)

$TC: O(\log_5 n)$

# Topic : Analysis of Algorithms

Generalized from

```
for (i=1; i<=n;i=i*b)
{
        print (i)
}
```

$TC = O(\log_b n)$

# Topic : Analysis of Algorithms

76%

#Q.
```
Algo AJ(n)
{
    for (j=1, j <= n/2 ; j++)
    {
        print(j)
    }

    i=0
    while (i<=n)
    {
        print(i)
        i=i+3
    }
    AJ2(n)
}
```

→ O(n)

→ O(n)

→ O(log n)

A. $O(n)$

B. $O(\log_3 n)$

C. $O(n^2)$

D. $O(n\log_3 n)$

# Topic : Analysis of Algorithms

#Q.

```
For(i=1; i<=n; i++)
{
    for (j=1;j<=n; j++)
    {
        for(k=n/2 ; k<=n; k=k+n/2)
        {
            print("AJ")
        }
    }
}
```

$i = 1 \to n$ ⟶ $O(n)$

$j = 1 \to n$ ⟶ $O(n)$

$\Rightarrow O(n^2)$

$k = n/2$

$k = n/2 + n/2 = n$

$k = 3n/2$

$\to O(1)$

85%

43%

| | |
|---|---|
| A | $O(n)$ |
| B | $O(n^2)$ |
| C | $O(n^3)$ |
| D | $O(n^2 \log n)$ |

# Topic : Analysis of Algorithms

6.

```
i=1
while (I <=n)
{
        i=i*5
            print (i)
}
```

$\hookrightarrow O(\log_5 n)$

# Topic : Analysis of Algorithms

7.

```
J = n
while (j >0)
{
        j = j/5
            print(j)
}
```

$j = n, n/5, n/5^2, n/5^3 \ldots n/5^k$

$TC = O(\log_5 n)$

$\frac{n}{5^k} = 1$

$5^k = n$

$k = \log_5 n$

# Topic : Analysis of Algorithms

8.

```
C =0
for (i=1; i<=n; i=i+2)
{
for (j=1; j<=m; j=j*2)
{
        c = c+1
}
}
```

$\rightarrow O(n)$

$\rightarrow O(\log_2 m)$

$T(n) = O(n * \log_2 m)$

# Topic : Analysis of Algorithms

#Q.

```
C = 0
for (i=1; i< =n; i++)
{
    for (j=i,j<=n; j++)
    {
        c = c+1
    }
}
```

nested dependent

57.5 Y.

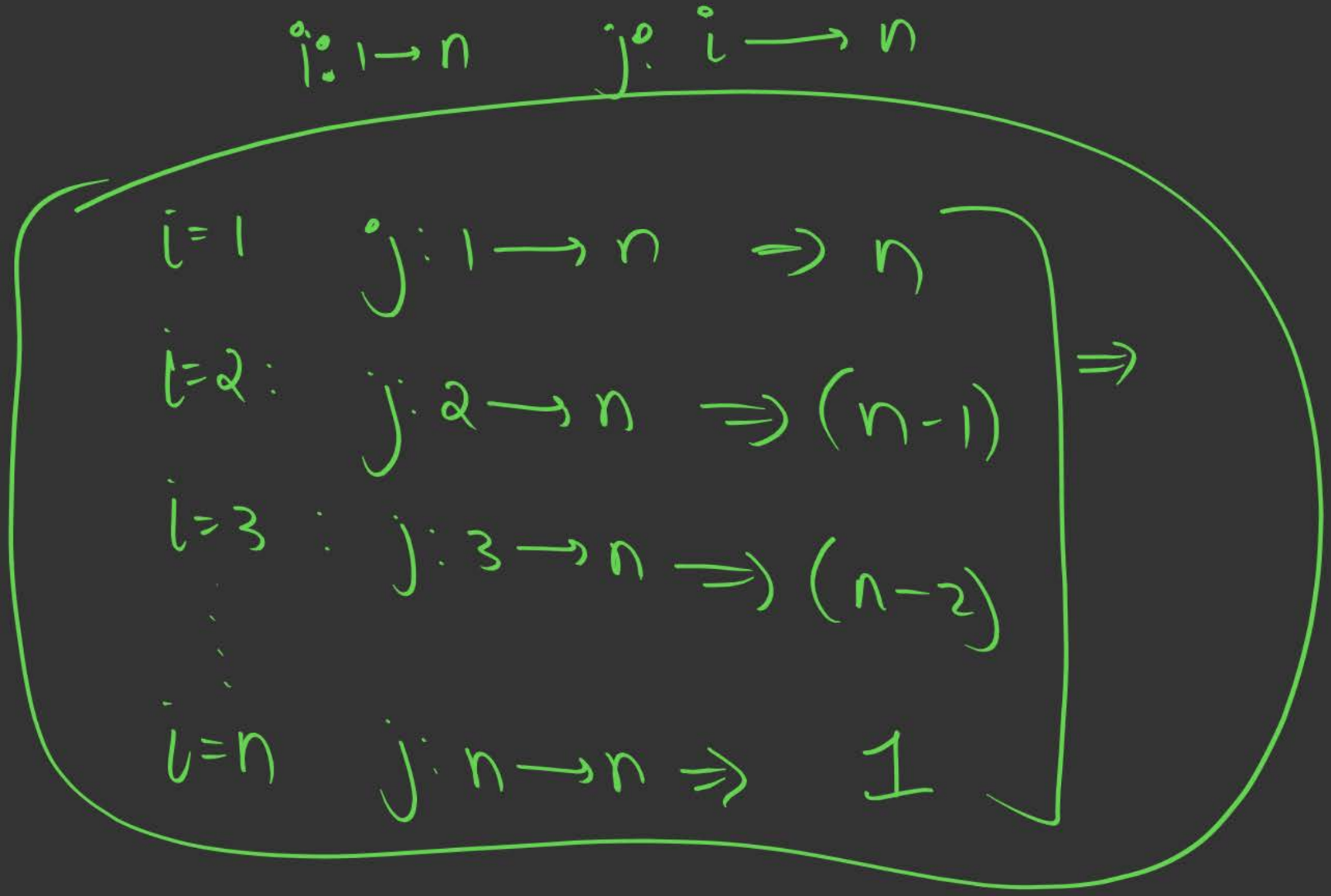
D

A. O(n)

B. $O(n^3)$

C. O(n logn)

D. $O(n^2)$


i: 1 → n
j: i → n
i=1
j: 1 → n ⇒ n
i=2
j: 2 → n ⇒ (n-1)
i=3
j: 3 → n ⇒ (n-2)
i=n
j: n → n ⇒ 1
⇒

$$\text{Total} = n + (n-1) \cdots 2 + 1$$

$$= \frac{n(n+1)}{2}$$

$$= O(n^2)$$

# Topic : Analysis of Algorithms

#Q.
```
a = 1, b =1
while (a<=n)
{
    b = b +1
    a = a + b
}
```

70%

32.5%

A $O(n)$

B $O(\sqrt{n})$

C $O(n^2)$

D $O(\log n)$

$a=1, b=1$

```
b=b+1
a=a+b
```

last itr

$b=1 \rightarrow b=2 \rightarrow b=3 \rightarrow \cdots \rightarrow b=k$

$a=1 \quad a=(1+2) \quad a=(1+2+3) \cdots a=(1+2+3 \cdots k)$

last itr $a \Rightarrow \frac{k(k+1)}{2} = n$

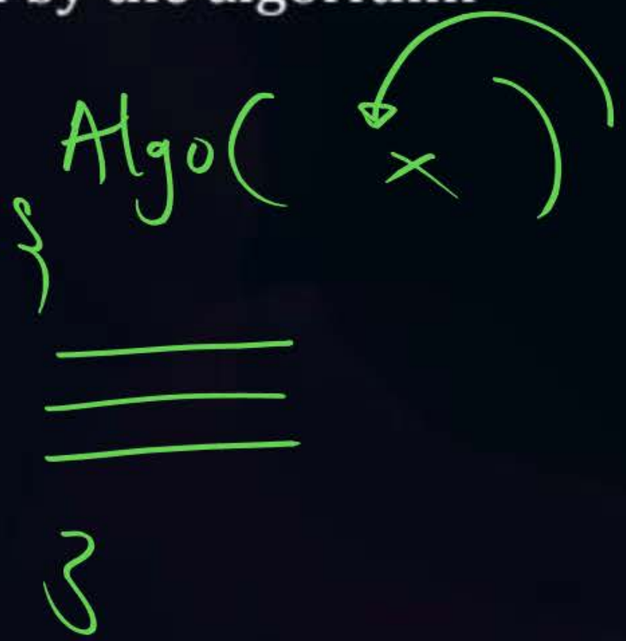
$\frac{k^2+k}{2} = n \Rightarrow k^2 \approx n$

$k \approx \sqrt{n}$ $\boxed{k=O(\sqrt{n})}$

# Space Complexity

# Topic : Space Complexity

We define the space used by an algorithm to be the number of memory calls (or words) needed to carry out the computation steps required to solve an instance of the problem excluding the space allocated to hold the input. In other word, it is only the work space required by the algorithm



# Topic : Space Complexity

Algo (input)

{

.....

}

Space complexity

Auxiliary space

(Additional space except the input)

# Topic : Space Complexity

**Eg.3.** Linear search

```
Algo AJ Search(A[n] , x , n)
{
    for (i=1; i<=n; i++)
        if (A[i] == x)
            return i
}
```

$T(n) = O(n)$

$SC = O(1)$

# Topic : Space Complexity

**Eg.4.** Algo sum AJ (A[n] , n)

```
{
    sum = 0
    for (i = 1; i<=n; i++)
    {
        sum = sum +A[i]
    }
}
```

TC = O(n)

SC = O(1)

# Topic : Space Complexity

**Eg.4.** Algo recursive sum (A, n)

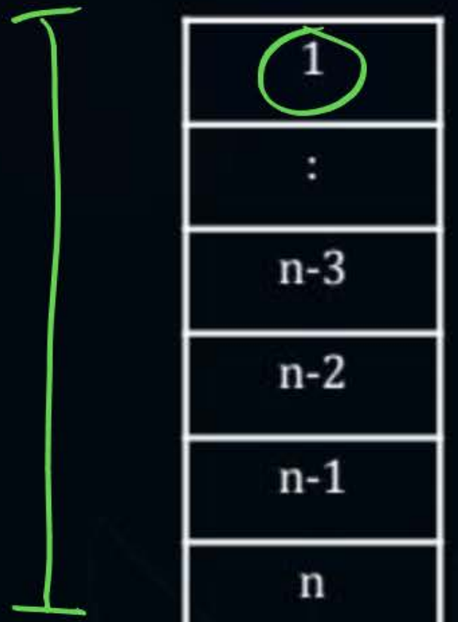
```
{
    it (n ==1)
        return A[n]
    else
        return (A[n] + recursive sum (A, n-1)
}
```

# Topic : Space Complexity

TC $\Rightarrow T(n) = T(n-1) + a$

$\Rightarrow O(n)$

Space complexity = O(n)







$k = n$

$S(n) = O(n)$

**Auxiliary Space**

Recursion stack

Space complexity

The max size of recursion stack at any time during recursion.

# Topic : Analysis of Algorithms

#Q. Algo AJ(n)

```
{
    if (n==1)
        return,
    else
        return AJ(n/2)
}
```

TC

SC

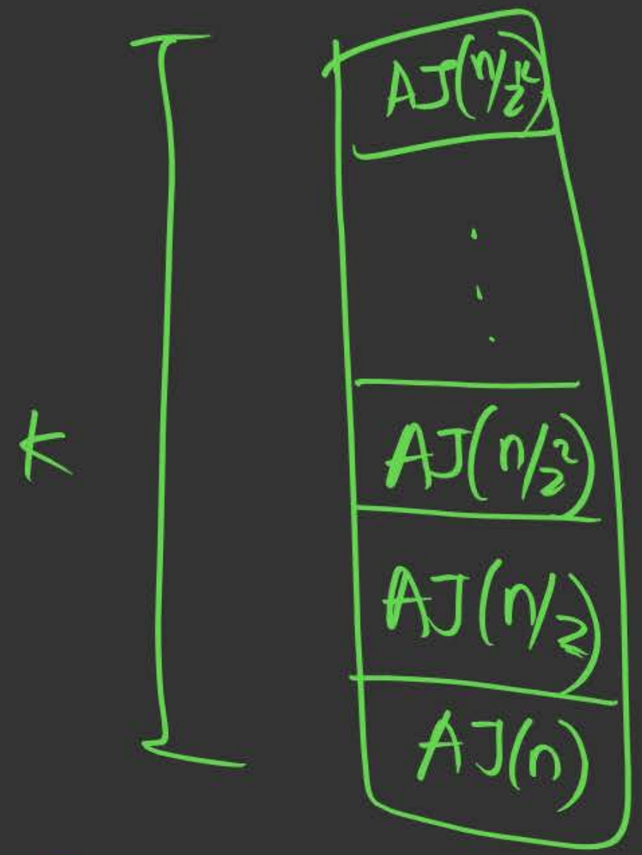
74%

A O(n)

B $O(n^2)$

C O(n logn)

D O(logn)




$$\frac{n}{2^k} = 1$$

$$2^k = n$$

$$k = \log_2 n$$

$$SC = O(\log_2 n)$$

# Topic : Analysis of Algorithms

#Q.

```
Algo AJ(n)
{
    for (i=1, i<=n;i=i+1)
    {
        for (j=1; j<=n; j=j+i)
        {
            print (i,j)
        }
    }
}
```

TC

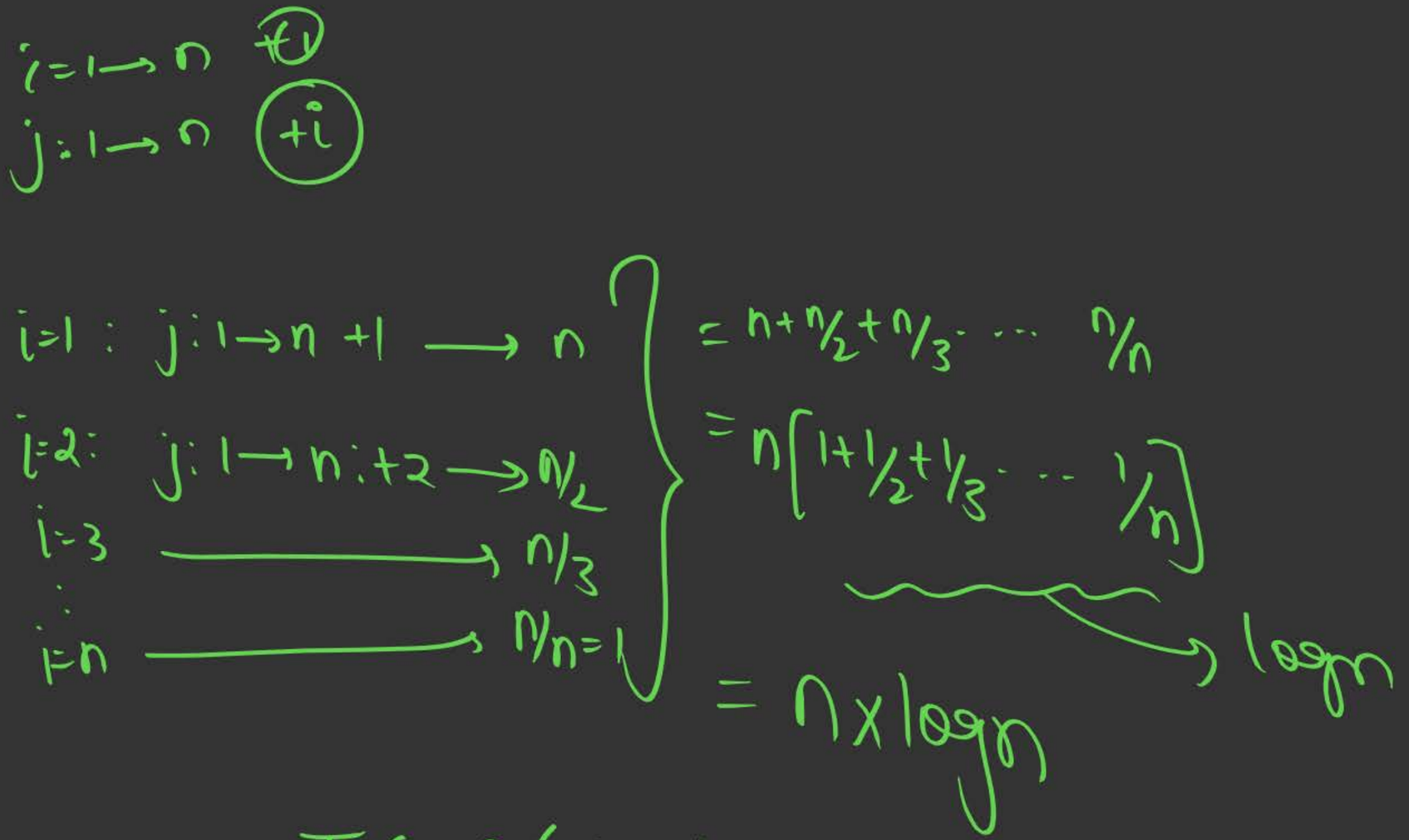
34 ✓

A O(n)

B $O(n^2)$

C O(n logn)

D O(logn)


i=1→n +1
j:1→n +i
i=1 : j:1→n +1 ⟶ n
i=2: j:1→n :+2 → n/2
i=3 ⟶ n/3
i=n ⟶ n/n=1
= n+n/2+n/3 ··· n/n
= n[1+1/2+1/3 ··· 1/n]
logn
= n x logn
TC=O(nlogn)

# Topic : Analysis of Algorithms

#Q.
```
Algo AJ(n)
{
    int i, j ,k = 0
    for (i=n/2; i <=n;i =i+1)
    {
        for (j=2; j <=n; j = j*2)
        {
            k= k+n/2
        {
    return (k)
    }
}
```

1) TC = ?

2) Return value = O(?)
   ↳ HW

O(n)

O(log n)

A. O(n)

B. $O(n^2 \log n)$

C. O(n log n)

D. $O(n^2 \log n)$

# Topic : Space Complexity

#Q. Value of K $\Rightarrow \frac{n\log n}{2} \times \frac{n}{2} \Rightarrow O(n^2 \log n)$

↓

HW Ans

# Topic : Analysis of Algorithms

HW P48

#Q. Consider functions function_1 and function_2 expressed in pseudocode as follow:

Let $f_1(n)$ and $f_2(n)$ denote the number of times the statement "x=x+1" is executed in function_1 and function_2, respectively. Which of the following statement is/are TRUE?

| Function_1 | Function_2 |
|---|---|
| While n > 1 do for i = 1 to n do<br>x = x + 1;<br>end for<br>n = [n/2];<br>end while | for i = 1 to 100*n do<br>x = x + 1;<br>end for |

**A** $f_1(n) \in \theta(f_2(n))$

**B** $f_1(n) \in 0(f_2(n))$

**C** $f_1(n) \in \omega(f_2(n))$

**D** $f_1(n) \in 0(n)$

# THANK - YOU